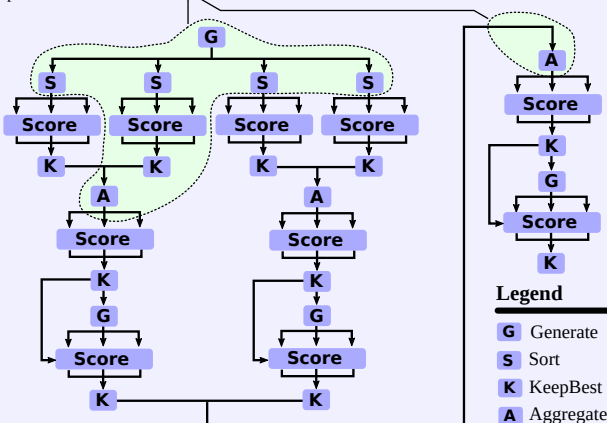


Note that this is an example graph decomposition. The structure of connections between all operations can be arbitrarily modified.



Input
64 numbers
1 4 6 2 4 ... 9 8 7 5 4

Generate(k) k = 1
Splitting into four 16-element chunks

The first Generate splits the 64-element input array into four 16-element chunks.

Partial solution
16 numbers
1 4 ... 4 3

Generate(k)
Sort k = 3

Partial solution
16 numbers
8 2 ... 1 3

Generate(k)
Sort k = 3

Partial solution
16 numbers
1 1 ... 4 2

Generate(k)
Sort k = 3

Partial solution
16 numbers
1 9 ... 5 4

Generate(k)
Sort k = 3

Sorting is implemented within the Generate operation. Here, k=3 means that, for each 16 element chunk, we generate three different sortings.

Score
Assess how well each sequence is sorted

Partial solution
16 numbers
1 2 ... 4 8

Score: 100%

Partial solution
16 numbers
1 2 ... 7 8

Score: 78%

Partial solution
16 numbers
1 1 ... 5 7

Score: 86%

How do we score?
To obtain the score, for every number 0 - 9, we get the difference between the input and the sorted list, and we sum all 10 values. Zero indicates correctly sorted. To show "the higher the better", we do $\max(\text{input_length} - \text{score}, 0)$

KeepBest(N)
Keep the best scored thoughts N = 1

Here, N=1 means that we maintain a single best sorting outcome out of the three input ones.

Aggregate(k)
Merge into a 32 element subarray k = 10

Partial solution
16 numbers
1 3 ... 4 6

Score: 97%

Partial solution
16 numbers
1 2 ... 4 8

Score: 100%

Aggregate(k)
Merge into a 64 element array k = 10

Here, k=10 means that we try 10 different aggregations of the two input 16-element subarrays.